DATABRICKS CERTIFIED DATA ENGINEER

Associate Exam — Comprehensive Study Guide

Chapter 3: Data Engineering in Databricks

Lakeflow Spark Declarative Pipelines, Medallion Architecture, Compute, CDC, and Production Deployment

EXAM FOCUS: This chapter is heavily tested. Pay close attention to: Streaming Tables vs Materialized Views vs Views, Medallion Architecture (Bronze / Silver / Gold), Data Quality Expectations (WARN / DROP / FAIL), CDC with AUTO CDC INTO (SCD Type 1 vs Type 2), Compute types (Serverless vs Classic), and Pipeline scheduling (Triggered vs Continuous).

1. The Databricks Lakehouse Architecture

Before diving into pipelines, you need the mental model that underpins everything in Databricks. The Lakehouse is not just a marketing term — it is a specific architectural decision with real design consequences for every pipeline you build.

1.1 What Is a Lakehouse?

A Lakehouse is an open data management architecture that combines the best of data lakes and data warehouses into a single, unified platform.

Traditional two-tier architectures kept raw storage (data lake) and analytical storage (data warehouse) separate, which created data duplication, stale copies, and governance gaps. The Lakehouse collapses these into one system.

Dimension	Data Lake	Data Warehouse	Lakehouse
Storage format	Raw files (Parquet, JSON, CSV)	Proprietary binary	Open formats (Delta Lake, Parquet, Iceberg)
ACID transactions	No	Yes	Yes (via Delta Lake)
Schema enforcement	Optional (schema-on-read)	Strict (schema-on-write)	Both supported
Cost	Low	High	Low (object storage)
Unstructured data	Yes	No	Yes
BI & ML on same data	Complex	ML difficult	Native support
Governance	Weak	Strong	Strong (Unity Catalog)

1.2 The Databricks Lakehouse Layers

The Databricks Lakehouse is structured in three distinct layers, stacked from bottom to top:

- **Optimized Storage:** Delta Lake (recommended), Apache Parquet, or Apache Iceberg. Data sits in your own cloud object storage (S3, ADLS, GCS) — Databricks never owns your data files.
- **Unified Governance:** Unity Catalog provides a single control plane for access control, auditing, data lineage, quality monitoring, and data discovery across all Databricks workspaces.
- **Lakeflow (Data Engineering Layer):** The end-to-end data engineering solution that sits on top of governance and storage. Includes Lakeflow Connect, Lakeflow Spark Declarative Pipelines, and Lakeflow Jobs.
- **Compute Engine:** Apache Spark + Structured Streaming, the industry-leading distributed processing engine, powers all transformations.

Refer to:

- What Is a Data Lakehouse? — Databricks Blog
- Databricks Lakehouse Architecture

2. Lakeflow: The Unified Data Engineering Platform

Lakeflow is Databricks' end-to-end data engineering solution. It reached General Availability in June 2025 and is now the default way to build data pipelines on the platform. Lakeflow has three pillars:

2.1 Lakeflow Connect

A set of efficient ingestion connectors that simplify bringing data from external sources into the lakehouse. Connect handles the plumbing so you can focus on transformation.

Supported source categories:

- **Cloud Storage:** Amazon S3, Azure ADLS Gen2, Google Cloud Storage — files of any format.
- **Message Queues / Message Buses:** Apache Kafka, Amazon Kinesis, Google Pub/Sub, Azure Event Hubs, Apache Pulsar.
- **Relational Databases:** SQL Server, PostgreSQL, MySQL, and other JDBC sources.
- **Software as a Service (SaaS):** Salesforce, Workday, and other enterprise applications.

Connect has two connector tiers: Managed Connectors (zero-code, UI-driven, minimum operational overhead) and Standard Connectors (more flexible, accessible from within pipeline code).

2.2 Lakeflow Spark Declarative Pipelines (SDP)

Formerly called Delta Live Tables (DLT), now rebranded as Lakeflow Spark Declarative Pipelines. This is a declarative framework for building batch and streaming ETL pipelines using SQL or Python.

Key insight: you declare WHAT you want (the transformation logic), and the pipeline engine figures out HOW to execute it — including dependency ordering, incremental processing, retries, and optimization.

- **Accelerated ETL development** — write SQL or Python, skip infrastructure boilerplate.
- **Built-in data quality** — expectations (constraints) are first-class citizens.
- **Unified batch and streaming** — same code handles both.

> *EXAM NOTE: The course material refers to this as 'Spark Declarative Pipelines' or 'SDP'. The old name 'Delta Live Tables (DLT)' and old SQL syntax (CREATE OR REFRESH STREAMING LIVE TABLE) still work for backward compatibility. On the exam you may see either name — treat them as equivalent.*

2.3 Lakeflow Jobs (Previously Workflows)

A workflow automation tool that orchestrates data processing workloads. Jobs coordinate multiple tasks within complex workflows, enabling scheduling, optimization, and management of repeatable processes.

Unlike Declarative Pipelines (which focus on data transformation), Jobs focus on orchestration — coordinating notebooks, Python scripts, SQL queries, Declarative Pipeline runs, and more.

Refer to:

- Data Engineering with Databricks
- What is Lakeflow Spark Declarative Pipelines — Databricks on AWS

3. Spark Declarative Pipelines vs. Lakeflow Jobs

This is one of the most commonly confused topics for Databricks beginners. The distinction is fundamental and exam-tested.

3.1 The Core Mental Model

Think of it this way:

- **Declarative Pipelines = Data Transformation Engine.** It answers the question: 'How does my data get from raw to refined?'
- **Lakeflow Jobs = Orchestration Engine.** It answers the question: 'When and in what order should things run?'

3.2 Detailed Comparison

Aspect	Spark Declarative Pipelines	Lakeflow Jobs
Purpose	ETL / data transformation	Workflow orchestration
Approach	Declarative — define WHAT, not HOW	Procedural — define order of tasks explicitly
Unit of work	Pipeline (contains flows, streaming tables, materialized views)	Job (contains tasks of many types)
Dependency management	Automatic — engine parses dependencies from code	Manual — you define task dependencies explicitly
Compute	Serverless Pipeline or Classic Pipeline	Serverless Jobs, Classic Jobs, All-Purpose, SQL Warehouse
Data quality	Built-in Expectations (CONSTRAINT)	Not built-in — must be coded manually
Scheduling	Built-in (triggered or continuous)	Built-in (trigger or schedule)
Can call a pipeline?	No — SDP is the pipeline	Yes — a Job can have a Pipeline Task that triggers SDP
Language	SQL or Python (.sql or .py files)	Notebooks, Python, SQL, JARs, dbt, etc.
Best for	Building Bronze/Silver/Gold medallion layers	End-to-end orchestration: ingest → transform → publish → notify

3.3 How They Work Together

Declarative Pipelines and Jobs are complementary, not competing. In a mature production setup, you typically see:

1. A Declarative Pipeline handles all incremental ingestion and transformation (Bronze → Silver → Gold).
2. A Lakeflow Job orchestrates the bigger picture: trigger the pipeline, run post-processing notebooks, send notifications, update a reporting table.

A Job can contain a Pipeline Task, which triggers a specific Declarative Pipeline. This is the recommended production deployment pattern.

3.4 Tasks vs. Pipelines — The Fundamental Difference

A Task is the atomic unit within a Job. Each task performs one specific action. Types of tasks include:

- **Notebook Task:** Runs a Databricks notebook.
- **Pipeline Task:** Triggers a Lakeflow Spark Declarative Pipeline.
- **Python Script Task:** Runs a Python script file.
- **SQL Task:** Runs SQL on a SQL Warehouse.
- **dbt Task:** Runs dbt models.
- **JAR Task:** Runs a compiled Spark JAR (classic compute only).

A Pipeline, by contrast, is a container for all your transformation logic — it is not a task itself, but it can be invoked by a task.

> *COMMON BEGINNER MISTAKE: Do not confuse 'creating a task' with 'creating a pipeline'. A pipeline defines your data flow (Bronze/Silver/Gold tables). A task in a Job calls that pipeline (or a notebook, or a script). If you need data quality checks, dependency tracking, and incremental processing — build a pipeline. If you need to orchestrate that pipeline with other work — wrap it in a Job with a Pipeline Task.*

Refer to:

- Lakeflow Jobs — Databricks on AWS
- Pipeline Task for Jobs — Databricks on AWS
- Procedural vs. Declarative Data Processing

4. Databricks Workspace UI: Navigation & Key Areas

Understanding the UI layout is essential for working efficiently and for answering exam questions about where specific features live.

4.1 Main Navigation Areas

- **Workspace:** Your file browser. Contains notebooks, SQL files, Python files, and folders. This is where your pipeline source code (.sql or .py files) lives.
- **Catalog (Unity Catalog):** Browse all data assets — catalogs, schemas, tables, views, volumes, and models. See lineage, permissions, and quality metrics.
- **Jobs & Pipelines:** Your orchestration hub. Create and manage Lakeflow Jobs and ETL Pipelines. This is where you navigate to create or run a Declarative Pipeline.
- **Compute:** Create and manage All-Purpose clusters, SQL Warehouses, and Instance Pools. View serverless compute status.
- **SQL Editor:** Run ad hoc SQL queries against tables, views, and SQL Warehouses.
- **Queries & Dashboards:** Saved SQL queries and BI dashboards built directly in Databricks.
- **Marketplace:** Browse and install data sets, notebooks, and solutions from partners.

4.2 Creating a Spark Declarative Pipeline (Two Methods)

Method 1 — From Workspace:

3. Click the (i) options button in the Workspace navigation bar.
4. Select Create.
5. Choose ETL Pipeline.

Method 2 — From Jobs & Pipelines:

6. Click Jobs & Pipelines in the left sidebar.
7. Click Create.
8. Select ETL Pipeline.

Both paths open the same Pipeline creation flow. You define your pipeline name, source code location, compute type, and optional configuration parameters.

4.3 The Multi-File Pipeline Editor

As of the Data & AI Summit 2025, Spark Declarative Pipelines feature a new multi-file editor with significant productivity improvements. Key features:

- **Pipeline Asset Browser:** Left panel showing all .sql and .py files belonging to the pipeline. You can edit multiple files without leaving the pipeline UI.
- **Multi-File Code Editor:** Edit each file directly in the browser with syntax highlighting.
- **Pipeline-Specific Toolbar:** Quick access to Start, Stop, Validate (dry run), and Full Refresh actions.
- **Interactive DAG (Directed Acyclic Graph):** Visual representation of your pipeline's tables and their dependencies. Click any node to inspect it.
- **Data Previews:** Inspect data samples from intermediate tables without running a full pipeline.
- **Execution Insights Panel:** See row counts, duration, errors, and warnings per table after each run.

- **Dry Run / Validation:** Validate syntax and dependency logic without executing the pipeline, saving time and cost.

Recommended file types for pipeline code: .sql and .py files (notebooks are also supported but .sql/.py are preferred for cleaner version control).

Refer to:

- Develop and debug ETL pipelines with the Lakeflow Pipelines Editor — Databricks on AWS

5. Unity Catalog — Deep Dive

Unity Catalog (UC) is Databricks' unified data governance layer. It sits between your compute and your storage, controlling everything about who can access what data and providing full auditability.

5.1 Unity Catalog's Three-Level Namespace

All data assets in Unity Catalog follow a three-level naming hierarchy:

```
catalog_name.schema_name.object_name

Example:  main.bronze_db.orders_bronze
          main.silver_db.orders_silver
          main.gold_db.gold_orders_by_date
```

- **Catalog:** The top-level container. An organization might have catalogs like 'dev', 'staging', 'prod', or one per business unit.
- **Schema (Database):** Within a catalog. Groups related tables and views. In the course project, each pipeline layer (bronze, silver, gold) gets its own schema.
- **Objects:** Tables, Views, Volumes, Functions, Models — all live inside schemas.

5.2 Unity Catalog Object Types

Unity Catalog governs several types of objects. The managed vs external distinction applies to tables and volumes only.

Tables

Type	Who Controls Storage?	Format	Best For	Governance
Managed Table	Databricks (Unity Catalog)	Delta Lake (always)	New tables, standard pipelines	Full — access, lineage, auditing, lifecycle
External Table	You / your cloud account	Delta, Parquet, CSV, JSON, etc.	Existing data already in cloud storage, multi-system access	Access control only — storage lifecycle is yours
Streaming Table (SDP)	Databricks (managed Delta)	Delta Lake	Incremental ingestion layers in a pipeline	Full — also tracks pipeline lineage
Materialized View (SDP)	Databricks (managed Delta)	Delta Lake	Pre-computed aggregations, gold layer outputs	Full — auto-refreshed by pipeline
View	N/A (virtual — no stored data)	N/A	Logical layer on top of other tables	Access control on the view

KEY INSIGHT: Databricks RECOMMENDS managed tables for most use cases. Use external tables only when: (1) data already exists in cloud storage that you cannot or do not want to move, or (2) other non-Databricks systems also need direct write access to the

underlying files. Managed tables get full performance optimizations and new Databricks features first.

Volumes — For Non-Tabular Data

Volumes are Unity Catalog objects that govern non-tabular data — files of any format (images, JSON, CSV, logs, PDFs, Parquet, etc.) stored in cloud object storage. Volumes are for path-based file access; use tables for tabular, query-able data.

Type	Who Controls Storage Location?	Best For
Managed Volume	Databricks chooses the path automatically within your cloud account	Simplest option. No need to manage cloud credentials or paths. Use for new Databricks-centric workloads.
External Volume	You specify a path in a pre-defined External Location	Add UC governance to existing data already at a specific cloud path. Use when data is shared with non-Databricks systems or when you need a specific folder location (e.g., raw landing zone).
My Files (Beta)	Per-user volume, automatically provisioned	Personal or exploratory work without creating catalog/schema/volume first.

In the course project, volumes are used to store source JSON files (e.g., /Volumes/dbacademy/ops/labuser/orders) that the pipeline reads with read_files().

5.3 Unity Catalog Governance Capabilities

- **Fine-grained access control:** Grant/revoke permissions on catalogs, schemas, tables, volumes, and even specific columns.
- **Data lineage:** Automatically tracks how data flows between tables across pipelines, notebooks, and jobs.
- **Audit logs:** Every data access and modification is logged for compliance.
- **Quality monitoring:** Monitor table health, freshness, and schema changes.
- **Data discovery:** Search for tables, annotate with tags and descriptions, and explore the catalog.

Refer to:

- What is Unity Catalog? — Databricks on AWS
- Managed versus external assets in Unity Catalog
- What are Unity Catalog volumes? — Databricks on AWS
- Unity Catalog best practices — Databricks on AWS

6. Data Ingestion Pipelines & the Medallion Architecture

The medallion architecture is the foundational design pattern for organizing data in a lakehouse. Every Databricks pipeline you encounter — including the course project — follows this pattern.

6.1 What Is the Medallion Architecture?

The medallion architecture organises data into three progressive quality layers:

Layer	Also Called	Data Quality	What Happens Here	Table Type in SDP
Bronze	Raw / Landing Zone	Raw — as-is from source	Ingest raw data with minimal transformation. Sometimes add metadata columns (processing_time, source_file). Schema may be loose.	Streaming Table (append-only)
Silver	Cleaned / Conformed	Cleaned, validated, typed	Apply transformations: type casting, null handling, deduplication, joins with reference data. Data expectations (CONSTRAINT) are applied here.	Streaming Table (append-only, incremental)
Gold	Business / Aggregated	Business-ready	Aggregate, summarise, and enrich. Produce business-level metrics, KPIs, and curated datasets. Optimised for downstream consumption.	Materialized View (aggregations)

6.2 Why Does the Course SQL File Create Bronze, Silver, and Gold Sequentially?

This is a question many students have. Here is the precise explanation:

When you write your pipeline SQL file, you define all three layers in a single file (or multiple files). The pipeline engine does NOT execute them in the order they appear in the file. Instead:

9. **Automatic dependency parsing:** The engine reads all CREATE OR REFRESH STREAMING TABLE and CREATE OR REFRESH MATERIALIZED VIEW statements and builds a dependency graph.
10. **Topological ordering:** It determines the correct execution order automatically. Bronze must run first (reads from source), then Silver (reads FROM STREAM bronze), then Gold (reads from silver).
11. **Execution:** The pipeline runs the tables in dependency order, not file order.

You write Bronze → Silver → Gold in the file because it is logical for a human to read. But even if you put them in reverse order in the file, the pipeline would still run correctly by parsing the FROM STREAM references.

> *EXAM TIP: 'Pipeline dependencies are parsed automatically — code order is NOT required.' This is a key differentiator of Declarative Pipelines vs procedural code.*

6.3 How Data Flows Through the Pipeline (Step by Step)

Run 1 — Fresh Pipeline:

12. Bronze Streaming Table ingests ALL data from the raw source (e.g., reads all files from a cloud storage volume using read_files()).
13. Silver Streaming Table transforms ALL records from the bronze table, applies expectations, and appends them.
14. Gold Materialized View re-computes the full aggregation from the silver table.

Run 2 — Incremental Update (new data has arrived):

15. **Bronze:** Uses Auto Loader (checkpoint-based tracking) to read ONLY the new files added since Run 1. Appends new rows. Old data is NOT re-read.
16. **Silver:** Uses the STREAM keyword on bronze — only transforms the NEW rows added to bronze in this run. Appends new rows to silver.
17. **Gold:** A Materialized View processes all current silver data, applying incremental refresh (on Serverless compute) or a full recompute where needed. Always reflects current state.

This incremental processing approach is the core reason to use Declarative Pipelines — no manual checkpoint management, no re-processing old data, and no wasted compute.

6.4 Batch vs. Streaming Ingestion

Mode	How it Works	When to Use
Batch Ingestion	Processes large volumes of data that arrive all at once (e.g., nightly file drops). Triggered manually or on schedule.	Historical backfills, nightly loads, infrequent bulk updates.
Streaming Ingestion	Handles data arriving continuously in small increments. Processes in near real-time using Structured Streaming.	Real-time dashboards, event-driven pipelines, IoT, Kafka/Kinesis feeds.

Declarative Pipelines support BOTH modes with the same code. You simply choose the compute mode (Triggered vs Continuous) and the engine adapts.

Refer to:

- Load data with Lakeflow Declarative Pipelines — Databricks on AWS
- What is Lakeflow Spark Declarative Pipelines — Databricks on AWS

7. Dataset Types in Spark Declarative Pipelines

Within a Declarative Pipeline, you work with four dataset types. Understanding their differences is critical for exam success.

7.1 Streaming Table (ST)

A Streaming Table is a managed Delta table with additional support for streaming or incremental data processing. It only processes NEW data since the last run.

Key characteristics:

- Supports both batch and streaming for incremental data processing (processes each file or record exactly once).
- Each refresh APPENDS new data — it never re-processes old data.
- Use the STREAM keyword in the FROM clause to read from another streaming table incrementally.
- Use FROM STREAM read_files(...) to ingest from cloud storage (invokes Auto Loader).
- File names are guaranteed to be read only once — no duplicates.
- SQL syntax: CREATE OR REFRESH STREAMING TABLE

Example — Bronze Layer (ingesting from files):

```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze AS
SELECT
  *,
  current_timestamp() AS processing_time,
  _metadata.file_name AS source_file
FROM STREAM read_files(
  '${source}/orders',
  format => 'JSON'
);
```

Example — Silver Layer (reading from another streaming table):

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver AS
SELECT
  order_id,
  timestamp(order_timestamp) AS order_timestamp,
  customer_id,
  notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

The STREAM keyword on the FROM clause tells the pipeline to read only the new rows appended to orders_bronze since the last run — not all historical rows.

Refer to:

- Load data with Lakeflow Declarative Pipelines — Databricks on AWS

7.2 Materialized View (MV)

A Materialized View is a managed Delta table that stores pre-computed query results. It is always refreshed to reflect the current state of upstream data.

Key characteristics:

- Processes records as needed to return accurate results for the CURRENT data state.
- Refreshed automatically by the pipeline when upstream tables change.
- Use without the STREAM keyword in the FROM clause — reads all current data from the source table.
- Can use incremental refresh (Serverless compute only) — a cost-based optimizer determines whether to incrementally update or fully recompute.
- Ideal for aggregations, joins across datasets, pre-computing slow queries, and caching frequently used results.
- Can be used ANYWHERE in the pipeline, not just the gold layer.
- SQL syntax: CREATE OR REFRESH MATERIALIZED VIEW

Example — Gold Layer aggregation:

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date AS
SELECT
  date(order_timestamp) AS order_date,
  count(*) AS total_daily_orders
FROM 2_silver_db.orders_silver
GROUP BY date(order_timestamp);
```

EXAM NOTE (Q2 2025): As of Q2 2025, any Materialized View that uses expectations (CONSTRAINT) is ALWAYS fully refreshed — incremental refresh is not applied when expectations are present.

Refer to:

- Incremental refresh for materialized views — Databricks on AWS

7.3 Temporary View

A Temporary View is an intermediate, virtual table that exists only during the pipeline's execution. It stores no physical data.

Key characteristics:

- Persists only for the lifetime of the pipeline run.
- NOT registered as an object in Unity Catalog — invisible to end users.
- Useful for breaking up complex transformations into readable intermediate steps.
- SQL syntax: CREATE TEMPORARY VIEW

```
CREATE TEMPORARY VIEW cleaned_orders AS
SELECT order_id, customer_id
FROM 1_bronze_db.orders_bronze
WHERE order_id IS NOT NULL;
```

Refer to:

- CREATE TEMPORARY VIEW (Lakeflow Declarative Pipelines) — Databricks on AWS

7.4 View

A View (standard, non-temporary) constructs a virtual table registered in Unity Catalog. It stores no physical data.

Key characteristics:

- Registered as a UC object — available after the pipeline runs.
- SQL syntax: CREATE VIEW
- LIMITATIONS: (1) The pipeline must be a Unity Catalog pipeline. (2) Views CANNOT contain streaming queries and CANNOT be used as a streaming source for another table.

7.5 Quick Comparison: All Four Dataset Types

Type	Stores Data?	UC Registered?	Lifetime	Incremental?	Streaming Source?
Streaming Table	Yes (Delta)	Yes	Permanent	Yes (append-only)	Yes (use STREAM keyword)
Materialized View	Yes (Delta)	Yes	Permanent	Where possible (Serverless)	No
Temporary View	No (virtual)	No	Pipeline run only	No	No
View	No (virtual)	Yes	Permanent	No	No

7.6 Deprecated DLT Syntax vs. Current Syntax

Current (SDP) Syntax	Deprecated (DLT) Syntax
CREATE OR REFRESH STREAMING TABLE	CREATE OR REFRESH STREAMING LIVE TABLE
CREATE OR REFRESH MATERIALIZED VIEW	CREATE OR REFRESH LIVE TABLE
CREATE VIEW / CREATE TEMPORARY VIEW	CREATE LIVE VIEW / CREATE TEMPORARY LIVE VIEW

The semantics are identical. Old syntax is still supported for backward compatibility, but the new syntax should be used for all new pipelines.

Refer to:

- CREATE VIEW (Lakeflow Declarative Pipelines) — Databricks on AWS

8. Course Project Overview: End-to-End Pipeline

The course project builds a production-ready, modular pipeline using Lakeflow Spark Declarative Pipelines. It ingests order, status, and customer JSON files from cloud storage and processes them through three separate flows within a single pipeline.

8.1 Flow 1: Orders

- **orders_bronze (Streaming Table):** Ingests orders JSON files from cloud storage.
- **orders_silver (Streaming Table):** Applies type casting and transformations to orders_bronze.
- **gold_orders_by_date (Materialized View):** Aggregates total orders by date from orders_silver.

8.2 Flow 2: Status (Join + Filter)

- **status_bronze (Streaming Table):** Ingests status JSON files.
- **status_silver (Streaming Table):** Transforms status_bronze.
- **full_order_info_gold (Materialized View):** Joins orders_silver and status_silver to produce a unified order view.
- **cancelled_orders (Materialized View):** Filters full_order_info_gold for cancelled orders.
- **delivered_orders (Materialized View):** Filters full_order_info_gold for delivered orders.

8.3 Flow 3: Customers — Change Data Capture (CDC)

- **customers_bronze (Streaming Table):** Ingests customer JSON files.
- **customers_bronze_clean (Streaming Table):** Filters and formats incremental inserts, updates, and deletes.
- **type1_customers_silver (Streaming Table with CDC):** Uses AUTO CDC INTO with STORED AS SCD TYPE 1 to overwrite customer data with the latest values.

WHY THREE SEPARATE FLOWS IN ONE PIPELINE? All three flows share the same compute and run within the same pipeline execution. They can cross-reference each other (Flow 2 joins with Flow 1's silver table). This is more efficient than three separate pipelines and allows the DAG to show all dependencies together.

9. Common Pipeline Settings

Pipeline settings are accessed via the gear icon in the pipeline editor. The three most important are Compute, Code Assets, and Configuration.

9.1 Compute Setting

Defines the resources where your pipeline runs. Two main options:

- **Serverless (RECOMMENDED):** Databricks manages all infrastructure. Optimises cost while maintaining performance. Enables incremental refresh for Materialized Views. Supports Serverless Performance Optimized mode for time-sensitive workloads. No need to manage or provision clusters.
- **Classic (Fixed-Size Clusters):** You or your workspace admin provisions a cluster. Users need permission to create compute. Enhanced autoscaling is enabled by default, which adjusts cluster size based on workload volume.

9.2 Code Assets Setting

Controls what code files your pipeline executes:

- **Pipeline Root Folder:** The top-level folder containing your pipeline project. All relevant files are automatically included. Supports Git folders for version control.
- **Source Code Section:** Explicitly list specific subfolders or files (.py, .sql, notebooks) to include. More granular control.

9.3 Configuration (Parameters)

A pipeline's configuration is a map of key-value pairs used to parameterise your pipeline code. Benefits:

- Improves code readability and maintainability by centralising environment-specific values.
- Enables reuse of the same pipeline code across dev, staging, and prod environments by changing only the configuration.

In your SQL code, reference a parameter using ${key-name} syntax:

```
-- In pipeline settings, define: source = /Volumes/dbacademy/ops/labuser

CREATE STREAMING TABLE bronze AS
SELECT * FROM STREAM read_files(
  '${source}/orders',
  format => 'JSON'
);
```

9.4 Other Pipeline Settings

Setting	Purpose
Default Location for Data Assets	Set the default catalog and schema where tables are created if not specified in the SQL.
Environment Dependencies	A pip requirements.txt-style file listing Python library dependencies for the pipeline.

Tags	Key-value tags for organising, categorising, and filtering pipelines. Useful for cost allocation.
Budget	Define cost controls and spending limits.
Advanced Settings	Options like publish event log to metastore, photon enablement, and pipeline mode.

Refer to:

- Configure a serverless pipeline — Databricks on AWS

10. Ensure Data Quality with Expectations (Constraints)

One of the most powerful features of Spark Declarative Pipelines is built-in data quality enforcement through Expectations. These are declarative rules that validate data row-by-row during pipeline processing.

10.1 Syntax

```
CONSTRAINT <constraint-name> EXPECT (<column-condition>) [ON VIOLATION <action>]
```

Expectations are defined inside the table definition, before the AS SELECT clause.

10.2 The Three Violation Actions

Action	What Happens on Failure	Effect on Pipeline	Use Case
WARN (default — no ON VIOLATION clause)	Invalid rows ARE written to the target table. Warning is logged.	Pipeline continues normally. Logs include counts of valid vs invalid records.	Monitoring and visibility — you want to see bad data but not block the pipeline.
ON VIOLATION DROP ROW	Invalid rows are DROPPED and NOT written to the target. Drop count is logged.	Pipeline continues. Bad rows are discarded.	Filtering out known bad records (e.g., records before a date threshold, invalid enum values).
ON VIOLATION FAIL UPDATE	The specific FLOW (table update) FAILS immediately. Other flows in the pipeline continue.	The failed flow requires manual intervention. Other flows are unaffected.	Critical data quality checks — e.g., NULL primary keys should never reach the target.

10.3 Complete Example with All Three Actions

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
  (
    -- WARN: logs invalid records but still writes them
    CONSTRAINT valid_notifications
      EXPECT (notifications IN ('Y','N')),

    -- FAIL: fails this flow if any timestamp is too old
    CONSTRAINT valid_date
      EXPECT (order_timestamp > '2021-01-01')
      ON VIOLATION FAIL UPDATE,

    -- DROP: silently discards rows with null customer_id
    CONSTRAINT valid_id
      EXPECT (customer_id IS NOT NULL)
      ON VIOLATION DROP ROW
  )
```

```
AS SELECT
  order_id,
  timestamp(order_timestamp) AS order_timestamp,
  customer_id,
  notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

10.4 How Expectations Are Applied (Row-by-Row Flow)

18. A row of data enters the pipeline flow.
19. The row is evaluated against all defined constraints.
20. If it PASSES all constraints: the row is kept and pipeline continues.
21. If it FAILS a constraint:
    - WARN: Row is still written; failure is logged.
    - DROP: Row is discarded; pipeline continues with remaining data.
    - FAIL: The specific flow fails immediately; manual intervention required; other flows continue.

EXAM NOTE (Q2 2025): Materialized Views that use expectations are ALWAYS fully refreshed when the pipeline runs. Incremental refresh is disabled when expectations are present on an MV.

11. Streaming Joins Overview

When joining streaming data in Declarative Pipelines, the join pattern you choose depends on whether one or both sides of the join are streaming.

11.1 Pattern 1: Stream-Snapshot Join (Streaming Table to Static Table)

Join a streaming table with a static/reference table that changes infrequently. The goal is to incrementally enrich new streaming data with reference information.

How it works: as new rows arrive in the streaming table, each new row is joined with ALL rows in the static table. Only the new streaming rows are processed — the static table is not re-read unnecessarily.

Use case example: enriching a transaction stream with country codes from a reference table.

```
CREATE OR REFRESH STREAMING TABLE enriched_transactions AS
SELECT t.*, c.country_name
FROM STREAM transactions_bronze t
JOIN country_reference c ON t.country_code = c.code;
```

11.2 Pattern 2: Streaming Table to Streaming Table (via Materialized View)

Join two streaming tables where both sides are continuously updated. A Materialized View is required to handle this join.

How it works: each time the pipeline runs, the MV processes all current data from both streaming tables and computes the join. Incremental refresh (on Serverless) applies where possible.

Use case example: joining orders_silver and status_silver to produce full_order_info_gold (as in the course project).

```
CREATE OR REFRESH MATERIALIZED VIEW full_order_info_gold AS
SELECT o.*, s.status
FROM 2_silver_db.orders_silver o
JOIN 2_silver_db.status_silver s ON o.order_id = s.order_id;
```

11.3 Pattern 3: Stream-Stream Joins (Advanced — Outside This Course's Scope)

True stream-stream joins, where only the NEW incoming data from each stream is joined (not all historical data). This pattern involves windowing logic and watermarks, and is covered in the advanced Databricks Streaming course.

Refer to:

- Streaming tables — Databricks on AWS
- Work with joins on Databricks — Databricks on Google Cloud

12. Message Queues — Role in Data Workflows

Message queues (also called message buses or event streaming platforms) are a critical component of real-time data architectures. Understanding their role relative to Databricks pipelines is important.

12.1 What Is a Message Queue?

A message queue is a durable, distributed system that buffers and delivers streams of events or messages between producers (systems that generate data) and consumers (systems that process data). They decouple the rate of data production from the rate of data consumption.

Common message queue systems:

- **Apache Kafka:** The most widely used. High-throughput, distributed, durable. Used in large-scale event streaming.
- **Amazon Kinesis:** AWS-managed event streaming service.
- **Google Pub/Sub:** GCP-managed messaging service.
- **Azure Event Hubs:** Azure-managed event streaming compatible with Kafka protocol.
- **Apache Pulsar:** Open-source alternative to Kafka.

12.2 Where Message Queues Fit in a Databricks Pipeline

Message queues appear in two places in the Databricks data architecture:

- **As a Source (ingestion side):** Lakeflow Connect ingests real-time events from Kafka, Kinesis, Pub/Sub, or Event Hubs into Bronze Streaming Tables. The pipeline reads from the queue as a continuous stream.
- **As a Sink (output side):** The Sink API (advanced feature) allows pipeline outputs to be written TO Kafka, Delta tables (across workspaces), or Azure Event Hubs — enabling real-time downstream consumption by other systems.

12.3 Why Use a Message Queue Instead of Direct File Ingestion?

Scenario	Better Choice
Batch files dropped to S3/ADLS nightly	Auto Loader (read_files) directly from cloud storage
Real-time events from IoT devices, clickstreams, financial ticks	Kafka/Kinesis → Streaming Table in Databricks
Microservices producing events that multiple consumers need	Kafka (fan-out: Databricks consumes, along with other systems)
Decoupling high-frequency producers from slower consumers	Kafka/Kinesis as buffer layer

Refer to:

- Lakeflow Connect — Databricks Documentation
- Use sinks to stream records to external services — Databricks on Google Cloud

13. Compute Types & Cost Optimization

Databricks offers multiple compute types, each suited to different workloads with different cost profiles. Choosing the right compute type is both an operational and cost decision — and it is tested in the exam.

13.1 All Compute Types in Databricks

Compute Type	Purpose	Lifecycle	Cost Profile
Serverless Compute for Notebooks	Run SQL and Python in notebooks on-demand	Auto-provisioned and terminated	Pay per query/cell execution. No idle cost.
Serverless Compute for Jobs	Run Lakeflow Jobs without managing infrastructure	Auto-provisioned per job run	Pay per job execution duration. No idle cost.
Serverless Pipelines	Run Lakeflow Spark Declarative Pipelines	Auto-managed by Databricks	Pay per pipeline execution. Recommended for most SDP workloads.
All-Purpose Compute (Classic)	Interactive development in notebooks. Shared by multiple users.	You create, start, stop, restart. Billed while running (even if idle).	Higher cost — compute runs whether or not you are using it. Risk of idle billing.
Jobs Compute (Classic Job Clusters)	Run automated jobs. Created automatically when job runs.	Auto-terminates when job completes. Cannot be restarted.	Efficient — only runs for the job duration. Lower cost than all-purpose for automation.
Instance Pools	Pool of idle, pre-warmed instances for fast cluster startup.	Created by admin. Instances are held ready.	Charges for idle instances in the pool. Useful when fast startup time is critical.
Serverless SQL Warehouses	Execute SQL queries — BI, reporting, SQL Editor.	Auto-start in 2-6 seconds, auto-stop after idle period.	Lowest cost SQL option. Rapid scaling, quick downscaling.
Classic SQL Warehouses (Pro/Standard)	Execute SQL queries — BI, reporting.	Manual or auto start/stop.	Higher than serverless. Slower startup (minutes).

13.2 Compute Types for Declarative Pipelines Specifically

Within a Declarative Pipeline (SDP), the compute choice is:

- **Serverless Pipeline (RECOMMENDED):** Databricks manages all infrastructure. Best for new pipelines. Enables incremental refresh for Materialized Views. Has an optional Serverless Performance Optimized setting for time-sensitive workloads.

- **Classic Pipeline (Fixed-Size):** Uses a Databricks-managed cluster of specified size. Enhanced autoscaling is enabled by default to right-size the cluster during execution.

13.3 Cost Reduction Strategies

Strategy	How It Reduces Cost
Use Serverless Compute	No idle billing. Auto-scales to zero. No cluster management overhead.
Use Jobs Compute instead of All-Purpose for automation	Jobs clusters terminate when done. All-purpose clusters can sit idle for hours.
Enable Enhanced Autoscaling on Classic Pipelines	Right-sizes the cluster during execution. Avoids over-provisioning.
Use Serverless SQL Warehouses	2-6 second startup. Quick downscaling when demand drops. Intelligent Workload Management (IWM) reduces costs.
Use Instance Pools only when startup latency matters	Pools save startup time but charge for idle instances — only beneficial if you need very fast, frequent cluster starts.
Triggered Pipeline mode vs Continuous	Triggered pipelines only run when data arrives or on schedule — no idle compute cost. Continuous mode keeps the cluster running, which is more expensive.
Incremental Refresh on Materialized Views (Serverless)	Avoids full recomputation of aggregations when only a small amount of new data arrives.
Budget controls in pipeline settings	Set spending limits to prevent runaway pipeline costs.

13.4 Photon Engine

Photon is Databricks' native vectorized query engine that accelerates SQL and DataFrame operations — especially joins, aggregations, and large data scans. Photon is available on Serverless SQL Warehouses and can be enabled on Classic compute. It significantly improves query performance at the same or lower cost.

Refer to:

- Compute — Databricks Documentation
- Configure compute for jobs — Databricks on AWS
- SQL warehouse types — Databricks on AWS
- What is Photon? — Databricks on AWS

14. Deploying a Pipeline to Production

Moving a pipeline from development to production requires three core operational tasks: scheduling, notifications, and monitoring.

14.1 Pipeline Scheduling Modes

Mode	Behaviour	Best For
Triggered	Pipeline runs once (processing data available at start), then STOPS. Can be triggered manually or on a recurring schedule (hourly, daily, etc.).	Batch workloads, nightly loads, cost-sensitive scenarios where the pipeline does not need to run continuously.
Continuous	Pipeline runs non-stop. Continuously monitors for new data and processes it in near real-time. Monitors dependencies and updates only when source data changes.	Streaming use cases, real-time dashboards, low-latency requirements. Higher compute cost — cluster stays running.

14.2 Email Notifications

Configure email notifications directly in the pipeline scheduling UI. You can choose to receive emails when:

- The pipeline starts,
- The pipeline succeeds, and/or
- The pipeline fails.

This allows you to monitor pipeline health without constantly checking the UI — critical for production operations.

14.3 Monitoring with the Pipeline Event Log

The pipeline event log is your primary tool for understanding what happened during a pipeline run. It captures:

- **Audit Logs:** Who started the pipeline, when, and what changed.
- **Data Quality Checks:** Results of every CONSTRAINT expectation — how many rows passed, were dropped, or caused a fail.
- **Pipeline Progress:** Status of each table update (running, complete, failed), row counts, and duration.
- **Data Lineage:** How data flowed and transformed through pipeline stages.

By default, the event log is stored as a hidden Delta table in the pipeline's default catalog and schema. You can publish it to a public Delta table for SQL analysis:

```
SELECT * FROM <catalog>.<schema>.<event_log_table_name>
```

To publish: enable the 'Publish event log to metastore' toggle in Advanced pipeline settings. Specify the catalog, schema, and table name for the event log.

Refer to:

- Query the event log — Databricks Documentation

15. Change Data Capture (CDC) in Declarative Pipelines

Change Data Capture is a technique for tracking and applying changes (inserts, updates, deletes) from a source system to a target system. It is fundamental to keeping lakehouse tables in sync with operational databases.

15.1 What Is CDC?

CDC captures a log of changes (inserts, updates, deletes) from a source system and applies those changes to a target table. The source data arrives as a stream of change events, each tagged with an operation type (INSERT, UPDATE, DELETE) and a sequence indicator (e.g., timestamp).

CDC is related to Slowly Changing Dimensions (SCD) — the strategy for HOW changes are stored in the target.

15.2 SCD Type 1 — Overwrite with Latest Value

In SCD Type 1, the target table always contains only the CURRENT state. There is no history.

- **UPDATE:** The existing row is overwritten with the new value.
- **DELETE:** The row is removed from the target.
- **INSERT:** A new row is added.

Use SCD Type 1 when historical values are irrelevant and only the current state matters — for example, a current customer address table where only today's address is needed.

15.3 SCD Type 2 — Historical Tracking with Versioning

In SCD Type 2, the target table preserves the full history of changes. Each version of a record is kept as a separate row.

Two metadata columns are added to track validity:

- **__START_AT:** Timestamp when this version became active.
- **__END_AT:** Timestamp when this version became inactive. NULL means the row is the CURRENT active version.

When a record is UPDATED: the old row's __END_AT is populated (making it inactive), and a new row is inserted with the new values and NULL __END_AT (making it the current active record). When a record is DELETED: its __END_AT is populated, but the row is KEPT in the table (marked inactive).

Use SCD Type 2 when historical data matters — for example, tracking all addresses a customer has ever had over time.

> *EXAM NOTE: SCD Type 2 is marked 'Outside the scope of this course' in the slides — it is covered in an advanced course. You should understand the concept but AUTO CDC INTO defaults to SCD Type 1.*

15.4 AUTO CDC INTO — Implementing CDC in Declarative Pipelines

The AUTO CDC INTO statement is Databricks' declarative, built-in way to implement CDC. It replaces the traditional batch-job pattern of using complex MERGE INTO SQL.

```
CREATE OR REFRESH STREAMING TABLE customers;
```

```
CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO customers          -- target table
FROM STREAM updates              -- source: stream of change events
KEYS (CustomerID)               -- primary key(s) to match rows
APPLY AS DELETE WHEN operation = 'DELETE'  -- flag deletes
SEQUENCE BY ProcessDate         -- event ordering column
COLUMNS * EXCEPT (operation)   -- columns to include in target
STORED AS SCD TYPE 1;           -- omit for Type 1 default
```

15.5 AUTO CDC INTO Clause Breakdown

Clause	Purpose
AUTO CDC INTO <table>	Specifies the target streaming table to apply CDC changes to.
FROM STREAM <source>	Specifies the source streaming table containing CDC change events.
KEYS (<columns>)	Primary key columns used to match rows between source and target.
APPLY AS DELETE WHEN <condition>	Defines the condition that identifies a row as a DELETE operation (e.g., operation = 'DELETE').
SEQUENCE BY <column>	Establishes the logical ordering of events. Critical for applying changes in the correct sequence when multiple changes exist for the same key.
COLUMNS * EXCEPT (<column>)	Specifies which columns to include in the target. Use EXCEPT to exclude the operation indicator column.
STORED AS SCD TYPE 1	Stores records using SCD Type 1 (overwrite). Default if omitted. Use STORED AS SCD TYPE 2 for history tracking.

KEY CHANGE: AUTO CDC INTO replaces the old APPLY CHANGES INTO syntax. Inserts and Updates are implicitly handled using the KEYS clause — no separate coding needed. Also new in 2025Q2: AUTO CDC FROM SNAPSHOT for processing changes from database snapshots (Public Preview).

Refer to:

- The AUTO CDC APIs: Simplify change data capture with Lakeflow Declarative Pipelines — Databricks on AWS
- AUTO CDC INTO (Lakeflow Declarative Pipelines) — Databricks on AWS

16. Additional Features Overview

These features extend Declarative Pipelines beyond what is covered in the core course. They are important for awareness (and sometimes tested) but less likely to require deep implementation knowledge in this certification.

16.1 Flows (Multi-Source to Single Target)

The Flows API allows you to add multiple flows that write to the SAME target table, creating a union of different data sources. Use case: ingesting from two different customer systems (e.g., CRM A and CRM B) into one unified customers table.

Refer to:

- Load and process data incrementally with Lakeflow Declarative Pipelines flows — Databricks on Google Cloud

16.2 Sinks (External Output)

The Sink API lets pipeline outputs be written to external systems OUTSIDE of Delta tables:

- **Kafka:** Publish pipeline results to a Kafka topic for downstream consumption.
- **Delta (cross-workspace):** Write to a Delta table in a different workspace.
- **Azure Event Hubs:** Stream pipeline results to Event Hubs for integration with Azure services.

Refer to:

- Use sinks to stream records to external services — Databricks on Google Cloud

16.3 Full Delta Lake Support

Lakeflow Pipelines have full native Delta Lake integration, enabling:

- **Liquid Clustering:** Automatically optimises file layout during pipeline runs for faster queries.
- **Row-Level Security & Column Masking:** Enforce fine-grained access controls directly on streaming and materialized tables.
- **Change Data Feed (CDF):** Capture row-level changes from streaming tables and use them in downstream audit logs or further CDC propagation.

16.4 Full Unity Catalog Support

- **Publish to multiple catalogs and schemas:** Write tables from a single pipeline to different catalogs (e.g., dev and prod).
- Read Streaming Tables and Materialized Views in Dedicated Access Mode.
- **Migrate from Hive metastore:** Clone an existing Hive-backed pipeline to create a Unity Catalog pipeline.

Refer to:

- Dedicated compute group access — Databricks on AWS
- Create a Unity Catalog pipeline by cloning a Hive metastore pipeline

16.5 Databricks Asset Bundles (DABs)

DABs enable you to programmatically validate, deploy, and run Databricks resources such as pipelines for CI/CD production workloads. With DABs, you define your Databricks resources as code (YAML configuration) and deploy them via CLI or CI/CD system.

Refer to:

- What are Databricks Asset Bundles? — Databricks on AWS
- Automated Deployment with Databricks Asset Bundles

17. Key Technical Terms Glossary

Term	Definition
Idempotency	An operation that can be applied multiple times and always produces the same result. In Databricks, pipeline runs are idempotent — re-running a triggered pipeline on the same data produces the same output without duplicates, because Auto Loader and checkpointing track what has already been processed.
Auto Loader	Databricks feature for incrementally ingesting files from cloud storage. Automatically tracks new files using checkpoints and reads each file exactly once. Invoked via FROM STREAM read_files() in a Streaming Table definition.
ACID Transactions	Atomicity, Consistency, Isolation, Durability. Delta Lake provides ACID guarantees on cloud object storage, enabling reliable reads and writes even in concurrent or failure scenarios.
DAG (Directed Acyclic Graph)	A graph of dependencies where edges point in one direction and there are no cycles. In Databricks pipelines, the DAG shows the dependency relationships between tables — Bronze feeds Silver feeds Gold, with no circular dependencies.
Incremental Processing	Processing only the NEW data since the last run, rather than reprocessing all historical data. Streaming Tables achieve this via Auto Loader checkpoints; Materialized Views achieve this via incremental refresh on Serverless compute.
Checkpoint	A persistent record of processing progress. Auto Loader writes checkpoints to track which files have been ingested, enabling exactly-once semantics across pipeline runs.
SCD (Slowly Changing Dimension)	A dimension table that tracks how attribute values change over time. Type 1 = overwrite (no history). Type 2 = versioned rows (full history preserved with start/end timestamps).
Schema Evolution	The ability of a table to automatically adapt when the source data schema changes (e.g., new columns added). Auto Loader supports schema evolution through configuration.
Photon	Databricks' native, open-source vectorized query engine written in C++. Accelerates SQL and DataFrame operations significantly compared to standard Spark. Available on Serverless SQL Warehouses and enabled Classic compute.
Liquid Clustering	A Delta Lake optimization that automatically reorganises table data files for faster query performance, replacing static partitioning strategies. Works during pipeline runs.
CDF (Change Data Feed)	A Delta Lake feature that records row-level changes (inserts, updates, deletes) to a table. Can be used to feed downstream CDC pipelines or audit systems.
DABs (Databricks Asset Bundles)	Infrastructure-as-Code framework for Databricks. Lets you define pipelines, jobs, and other resources in YAML and deploy them via CLI for CI/CD workflows.

Backfill Handling	Processing historical data that was missed or needs to be reprocessed. Declarative Pipelines handle backfills through Full Refresh — which re-ingests all source data and rebuilds tables from scratch.
Full Refresh	A pipeline run option that drops and recreates all tables from scratch, re-processing all source data. Use when schema changes, data corrections, or logic changes require a complete rebuild.

18. Common Beginner Mistakes to Avoid

18.1 Pipeline & Architecture

- **Confusing Declarative Pipelines with Lakeflow Jobs:** Pipelines are for data transformation (Bronze → Silver → Gold). Jobs are for orchestration (triggering pipelines + other tasks in sequence).
- **Assuming file order in the SQL script matters:** The engine parses dependencies automatically. You can define Gold before Bronze in the file and the pipeline will still run Bronze first.
- **Putting aggregations in a Streaming Table instead of a Materialized View:** Aggregations like COUNT(*) or SUM() require a Materialized View. Streaming Tables only support append operations.
- **Forgetting the STREAM keyword in Silver:** If you write FROM bronze_table (without STREAM), you will get a full recompute every run, not incremental processing. Always use FROM STREAM <streaming_table> to get incremental behavior.

18.2 Data Quality

- **Thinking FAIL fails the entire pipeline:** ON VIOLATION FAIL UPDATE only fails the specific flow (table update). Other flows in the pipeline continue running.
- **Not including expectations in the Silver layer:** Bronze should be raw (minimal validation). Silver is where you enforce data quality rules. Gold should already be clean.

18.3 Compute & Cost

- **Using All-Purpose compute for automated jobs:** All-Purpose clusters bill while running, even if idle. Use Job Compute (Classic) or Serverless for automation — they terminate when done.
- **Choosing Classic pipeline when Serverless is available:** Serverless pipelines are recommended for new development. They are cheaper to operate and enable incremental refresh for Materialized Views.
- **Running Continuous mode for batch workloads:** Continuous mode keeps the cluster running 24/7. Only use it for genuine low-latency streaming. Use Triggered mode with a schedule for batch workloads.

18.4 Unity Catalog & Views

- **Using CREATE VIEW in a pipeline and expecting it to work without Unity Catalog:** Standard VIEWs require a UC pipeline. Use CREATE TEMPORARY VIEW if you do not have UC or if the view is just an intermediate step.
- **Expecting Temporary Views to persist after the pipeline run:** Temporary Views exist ONLY during the pipeline run. They are not registered in UC and are not visible to end users or other pipelines.
- **Using External Tables when Managed Tables are better:** Default to managed tables. Use external tables only for data that already exists in specific cloud paths or that needs to be written by non-Databricks tools.

18.5 CDC Mistakes

- **Forgetting SEQUENCE BY:** Without SEQUENCE BY, CDC events may be applied out of order, resulting in incorrect data. Always specify a timestamp or sequence column.

- **Not specifying STORED AS SCD TYPE when history is needed:** The default is SCD TYPE 1 (overwrite). If you need history tracking, explicitly use STORED AS SCD TYPE 2.

19. Quick Reference — Exam Cheat Sheet

19.1 SQL Syntax Reference

```
-- Bronze Streaming Table (ingest from files with Auto Loader)
CREATE OR REFRESH STREAMING TABLE catalog.schema.bronze_table AS
SELECT *, current_timestamp() AS processing_time
FROM STREAM read_files('${source}/path', format => 'JSON');

-- Silver Streaming Table (incremental transform from bronze)
CREATE OR REFRESH STREAMING TABLE catalog.schema.silver_table AS
SELECT col1, cast(col2 AS timestamp) AS col2_ts
FROM STREAM catalog.schema.bronze_table;

-- Gold Materialized View (aggregation from silver)
CREATE OR REFRESH MATERIALIZED VIEW catalog.schema.gold_view AS
SELECT date(col2_ts) AS date, count(*) AS total
FROM catalog.schema.silver_table
GROUP BY date(col2_ts);

-- Data Quality Expectations
CREATE OR REFRESH STREAMING TABLE silver
  (
    CONSTRAINT c_warn   EXPECT (col1 IN ('Y','N')),
    CONSTRAINT c_drop   EXPECT (id IS NOT NULL)  ON VIOLATION DROP ROW,
    CONSTRAINT c_fail   EXPECT (ts > '2021-01-01') ON VIOLATION FAIL UPDATE
  ) AS SELECT ...

-- CDC with AUTO CDC INTO
CREATE OR REFRESH STREAMING TABLE customers;
CREATE FLOW cdc_flow AS
AUTO CDC INTO customers
FROM STREAM updates
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY process_date
COLUMNS * EXCEPT (operation)
STORED AS SCD TYPE 1;

-- Temporary View (intermediate logic, not UC-registered)
CREATE TEMPORARY VIEW intermediate_view AS
SELECT * FROM silver_table WHERE active = 1;

-- Standard View (UC-registered, no streaming)
CREATE VIEW catalog.schema.my_view AS
SELECT * FROM catalog.schema.gold_table;
```

19.2 Key Comparisons for the Exam

Question	Answer
Which dataset type is incremental (append-only)?	Streaming Table

Which dataset type auto-refreshes for current state?	Materialized View
Which dataset type is NOT registered in Unity Catalog?	Temporary View
Which dataset type cannot contain streaming queries?	VIEW (standard)
WARN expectation action — are rows written?	YES — rows are still written with a warning logged
DROP expectation action — does the pipeline fail?	NO — pipeline continues, bad rows are dropped
FAIL expectation action — does the whole pipeline fail?	NO — only the specific FLOW fails; other flows continue
Serverless vs Classic compute — which enables incremental MV refresh?	Serverless only
What keyword reads incrementally from a streaming table?	FROM STREAM <table_name>
What function reads files from cloud storage with Auto Loader?	read_files() — used inside FROM STREAM read_files(...)
SCD Type 1 — is history kept?	NO — old values are overwritten
SCD Type 2 — how to identify the current row?	__END_AT IS NULL = current active row
Triggered vs Continuous pipeline mode — which stops after each run?	Triggered
What replaces APPLY CHANGES INTO?	AUTO CDC INTO
What do pipeline dependencies require from the developer to set up?	Nothing — they are parsed automatically from the code
Where do you find pipeline execution metrics and lineage?	Pipeline Event Log

End of Chapter 3 Study Guide

Source material: Databricks Introduction to Data Engineering — Chapters: Data Engineering in Databricks, Lakeflow SDP Fundamentals, Building Lakeflow SDP.